Bayesian Neural Network

CJ Wessels - 25888773

March 23, 2026

Abstract

In this project, we compare the performance of a Bayesian Neural Network and a Neural Network, with a focus on their behaviour under limited training data. Both models are built separately and evaluated on the same training, validation, and testing sets using Recall, F_1 Score, training loss, and validation loss. We find that the classification performance of the two models is relatively similar on the *Breast Cancer Wisconsin* dataset. However, as the training set size decreases, the Neural Network begins to overfit, whereas the Bayesian Neural Network remains more robust to overfitting. Furthermore, the Bayesian Neural Network is able to quantify predictive uncertainty, providing additional insight into model confidence while maintaining better generalisation on smaller training sets.

1 Neural Networks

In this section, we present fundamental concepts related to Neural Networks (NNs) that will support our understanding of Bayesian Neural Networks (BNNs). A basic familiarity with neural networks is assumed.

1.1 Composition of Neural Networks

NNs are a powerful tool in machine learning and form the foundation of deep learning. NNs consist of interconnected nodes (neurons) that process data, learn patterns within the data and make predictions based on the learned patterns.

NNs are composed of an input layer, hidden (middle) layers and an output layer. Between each layer are weights and biases which determine the strength and influence of connections between layers. These parameters need to be optimised to produce a NN whose prediction ability is exemplary. At each layer in the NN a weighted sum is calculated as follows:

$$\begin{gathered} z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b \\ \text{or} \\ z = \mathbf{w}^T\mathbf{x} + \mathbf{b} \end{gathered} \tag{1.1}$$

where $[w_1, w_2, \ldots w_n] = \mathbf{w}$ are the weights, $[x_1, x_2, \ldots x_n] = \mathbf{x}$ are the inputs and b is the bias. Additionally, at each layer, an activation function is applied. An activation function is a mathematical function applied to the weighted sum (1.1) and decides whether the neuron should

be activated (fire) based on the answer after the activation function is applied to the weighted sum (1.1) at each layer. No specific activation functions will be discussed as this is outside the scope of this project.

1.2 Training and Evaluation of Neural Networks

In summary, a NN is trained by learning the optimal weights and biases for each layer in order to make predictions that align as closely as possible with the observed (true) data. The first phase of training is known as the forward pass, during which input is propagated through the network, from the input layer to the output layer, and the loss function measures the discrepancy between predicted and observed values. The second phase is the backward pass, in which the network moves from the output layer back to the input layer, updating the weights and biases based on the loss obtained during the forward pass. Although several optimisation methods exist for updating these parameters, a discussion of these methods is outside the scope of this project.

Training a neural network corresponds to estimating parameters θ of a conditional model $p(y|x, \theta)$. We find θ by maximising the likelihood. The likelihood represents the probability of the observed data given the model parameters. Mathematically, we define it as follows:

$$p(D|\theta) = \prod_{i=1}^{N} p(y_i|x_i, \theta), \tag{1.2}$$

where $D = \{(x_i, y_i)\}_{i=1}^{N}$ and provided that the data follows the independent and identically distributed (i.i.d) assumption. If the data follows (1.2) then, we can find θ to maximise the likelihood by following the next 2 equations:

$$\theta^*_{\text{MLE}} = \arg\max_{\theta} \left[\sum_{(x_i, y_i) \in D} \log p(y_i|x_i, \theta) \right], \tag{1.3}$$

or we can rewrite it in terms of the negative log-likelihood:

$$\theta^*_{\text{MLE}} = \arg\min_{\theta} \left[- \sum_{(x_i, y_i) \in D} \log p(y_i|x_i, \theta) \right]. \tag{1.4}$$

From (1.3) and (1.4), θ^*_{MLE} represents the estimated parameters (in this case, weights and biases) using Maximum likelihood estimation (MLE), y_i is the target output (label or value), and x_i denotes the input features.

Evaluation of a NN occurs after training and assesses how well the learned conditional model $p(y|x, \theta^*)$ predicts unseen data. When evaluating the NN on the testing data, this produces predictions, which can be compared with the observed data (y_{true}). In essence, evaluation of a NN measures how well this learned probability model (from the training) predicts unseen data.

2 Bayesian Neural Networks

Bayesian Neural Networks (BNNs) are a type of NN where the weights and biases are treated as random variables with prior distributions. BNNs can often perform better on small datasets because the prior distribution regularises the model and reduces overfitting. BNNs can quantify the uncertainty of the model's parameters (e.g. by computing their variance) which is especially relevant to smaller datasets. For small datasets a NN might overfit but Bayesian inference computes the posterior distribution over model parameters given the observed data. Consider the following equation called Bayes' Theorem which forms a foundation for Bayesian Inference:

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}. \tag{2.1}$$

From equation (2.1), $p(A|B)$ is the posterior (probability of A given B), $p(B|A)$ is the likelihood (probability of B given A), $p(A)$ is called the prior (initial belief of A before seeing data) and $p(B)$ is called the evidence (probability of observing B under any parameter choice). Equation (2.1) allows us to compute the posterior distribution of the parameters given the data.

2.1 Composition of Bayesian Neural Networks

As previously mentioned, the weights and biases of a BNN are represented as probability distributions. For example, consider the following 2 equations:

$$w \sim N(\mu_w, \sigma_w^2), \tag{2.2}$$

$$b \sim N(\mu_b, \sigma_b^2), \tag{2.3}$$

where the weights and biases are distributed according to a Gaussian distribution with a specific mean and variance. The main difference between a BNN and a standard NN is that the weights and biases are treated as random variables with distributions rather than fixed point estimates, but the other components remain the same.

2.2 Training and Evaluation of Bayesian Neural Networks

The training and evaluation of BNNs is very similar to NNs however, NNs find point estimations whereas BNNs aim to approximate the posterior distribution over the weights and biases. A method to find the optimal θ to maximise the posterior is to use Maximum A Posteriori Estimation (MAP). MAP is defined as follows:

$$\theta^*_{\text{MAP}} = arg \max_{\theta} p(\theta|D), \tag{2.4}$$

however, we can write equation (2.4) slightly differently using equation (2.1). Equation (2.4) becomes:

$$\theta^*_{\text{MAP}} = arg \max_{\theta} p(D|\theta)p(\theta), \tag{2.5}$$

where D is defined as in equation (1.2). In Equation (2.5), the $p(D)$ term does not appear because it is independent of θ. If the data follows the i.i.d assumption then, the MAP is calculated as follows:

$$\theta^*_{\text{MAP}} = arg \max_{\theta} \left[\log(p(D|\theta)) + \log(p(\theta)) + \text{constant}\right], \tag{2.6}$$

where the constant term is $-\log(p(D))$. Unfortunately, equations (2.4), (2.5) and (2.6) find point estimations and BNNs need distributions, furthermore, $p(\theta|D)$ is intractable making it near impossible to calculate it explicitly. Using variational inference enables the BNN to approximate the true posterior distribution using a tractable distribution (Talpini [2023]). We define variational inference as follows:

$$p(\theta|D) \approx q(\theta), \tag{2.7}$$

where $q(\theta)$ is the distribution that can approximate the posterior distribution, which is learned during training. To quantify the difference between $p(\theta|D)$ and $q(\theta)$, we use the Kullback-Leibler (KL) divergence, which is defined as the following:

$$\text{KL}(q(\theta)||p(\theta|D)) = \langle \log\left(q(\theta)\right) - \log\left(p(\theta|D)\right)\rangle_{q(\theta)}, \tag{2.8}$$

but because the posterior is unknown, in this case the posterior is replaced with the prior $p(\theta)$. Therefore, equation (2.8) becomes:

$$\text{KL}(q(\theta)||p(\theta)) = \langle \log\left(q(\theta)\right) - \log\left(p(\theta)\right)\rangle_{q(\theta)}. \tag{2.9}$$

BNNs use a different method when making predictions when compared to NNs. In particular, finding the posterior enables the BNN to perform probabilistic predictions as follows:

$$p(y|x, D) = \int_{\theta} p(y|x, \theta)p(\theta|D)d\theta, \tag{2.10}$$

where $p(y|x, D)$ is our prediction for y given the unseen data x and knowing the previous data D, $p(y|x, \theta)$ is our prediction for y given the unseen data x and knowing the parameters θ. These predictions can be compared with validation data to assess model performance and tune hyperparameters. Thereafter, the model can be evaluated against the testing data to acquire the final performance ability of the BNN. To quantify the performance, we minimise the evidence lower bound (ELBO) which ultimately minimises the loss when training and validating the model. We achieved this by equating the loss to the cross-entropy plus the KL divergence as seen in the next equation (Mittal [2025]):

$$\mathcal{L} = -\mathbb{E}_{q(\theta)}\left[\log p(D|\theta)\right] + \alpha \text{ KL}\left(q(\theta)\|p(\theta)\right), \tag{2.11}$$

where α is a term which controls the weight (or contribution) of the KL term in equation (2.11). For classification tasks, the negative log-likelihood corresponds to the cross-entropy loss.

The Cross-Entropy which is defined as the following (Hughes [2025]):

$$\text{CE} = -\sum_{k=1}^{K} y_k \log\left(p_k\right), \tag{2.12}$$

is the negative log-likelihood according to Mittal [2025]. In equation (2.12), y_k is the true probability of class k (typically k will either be 1 if correct or 0 when incorrect) and p_k is the predicted probability of class k.

2.3 A Problem Behind Probabilistic Predictions

Equation (2.10) enables the model to perform probabilistic predictions however, it is highly non-trivial to evaluate the predictive distribution. There are approaches that were proposed to overcome this roadblock. We present only a few here, but there are many possible approaches. Some approaches include (Talpini [2023], Duvenaud):

- Markov Chain Monte Carlo (MCMC)
- Monte Carlo (MC) Approximation
- Laplace Approximation

In this study, MC Approximation was implemented and the predictive distribution was approximated. The following equation explains the implementation:

$$p(y|x, D) \approx \frac{1}{N} \sum_{i=1}^{N} p(y|x, \theta_i), \quad \theta_i \sim q(\theta) \tag{2.13}$$

where the RHS represents the average prediction across multiple samples of the network parameters. MC Approximation offers a practical solution to this challenge, enabling the approximation of the predictive distribution.

3 The Dataset and the Task

The task of this project was to predict whether a patient had benign (non-cancerous) or malignant (cancerous) growth in their breast. The dataset employed in this project is known as the *Breast Cancer Wisconsin (Diagnostic)* dataset, and contains 569 instances and 32 features. Out of all these instances, 357 are benign and 212 are malignant. This section details the preprocessing done on the dataset and other metrics utilised to compare the different models.

3.1 Data Preprocessing

The dataset was loaded using the *scikit-learn* Python package, which facilitated the separation of the target feature from the training features. This dataset has no missing instances. With regards to outliers, none were removed because uncontrolled growths do vary with each individual. Consequently, all instances of growths were considered in the training of the BNN and NN models. The following figure is a snapshot of the columns in this dataset:

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal dimension',
       'radius error', 'texture error', 'perimeter error', 'area error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error', 'fractal dimension error',
       'worst radius', 'worst texture', 'worst perimeter', 'worst area',
       'worst smoothness', 'worst compactness', 'worst concavity',
       'worst concave points', 'worst symmetry', 'worst fractal dimension'],
      dtype='object')
```

Figure 1: The columns within the Breast Cancer dataset.

From Figure 1 it is shown that many of the columns contain error measurements and "worst" measurements (or the largest mean value). These columns may not contribute significantly in determining whether a growth was cancerous or not because, from Cleveland Clinic [2024] it says that a malignant tumour is typically spread out and grows faster than a benign tumour which is localised and grows slower. Therefore, the error measurements and "worst" measurements may not contribute significantly to the classification and keeping these columns could increase the training time of the models due to the higher dimensionality and computational complexity. Additionally, keeping these features could lead to the model overfitting and negatively affect its generalisation ability. The features kept for training were *mean radius*, *mean texture*, *mean perimeter*, *mean area*, *mean smoothness*, *mean compactness*, *mean concavity*, *mean concave points*, *mean symmetry* and *mean fractal dimensions*.

For further analysis of the dataset, the *Pandas* package was used, and the *.describe()* function provided an overview of the remaining data. Figure 2 displays the findings:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440

Figure 2: Results from *pandas.describe* method.

Figure 2 shows a range issue with some features. For example, take the column *mean perimeter* and *mean smoothness*, these two columns have different ranges. The mean value for the *mean perimeter* column is 91.969 whereas for the *mean smoothness* column the mean value is 0.0963. These are very different ranges and will affect our results on the training, validation and testing data. A solution to this problem is using *StandardScaler* from the *scikit-learn* package. This method is a feature scaling technique and is used to standardise the values of numeric features. The formula is as follows:

$$X_{\text{Scaled}} = \frac{X - \mu}{\sigma}, \tag{3.1}$$

where X_{Scaled} is the new scaled data, X is the original data, μ is the mean of X and σ is the

standard deviation (std) of X. This preprocessing technique will help to optimise the performance of the models.

3.2 Performance Metrics

To quantify the performance of the models and ultimately compare them, specific performance metrics were utilised in this project to achieve the necessary comparisons. In this project the Recall, F_1 Score and ROC curves with AUC were employed. The next two equations show how the Recall and F_1 Score were computed:

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{3.2}$$

where TP is the true positives (correctly classified benign) and FN (incorrectly classified benign).

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{3.3}$$

where Precision $= \frac{TP}{TP+FP}$ and FP is false positives (incorrectly classified malignant).

The Receiver Operating Characteristic (ROC) plots the False Positive Rate (FPR) against the True Positive Rate (TPR) and the Area Under Curve is the area under this ROC curve.

These three performance metrics were implemented for the following reasons:

- **Recall:** Measures how well the model can identify truly benign cases out of all benign classifications.
- F_1 **Score:** Harmonic mean between the precision and recall. Particularly useful when working with imbalanced datasets, as is the case here.
- **ROC and AUC:** Final check to quantify classification ability of models.

Because BNNs yield a predictive distribution, it is possible to compute the entropy, which serves as a measure of uncertainty and is defined as follows:

$$H(p) = -\left\langle \log\left(p(x)\right)\right\rangle. \tag{3.4}$$

Using equation (3.4) the uncertainty can be measured to quantify the confidence of the BNN when it makes predictions. These results are outlined in the Results section.

4 Results

In this section, the results of this project are presented as follows. The first subsection is our initial hyperparameter optimisation followed by the results of the BNN and NN using the optimal hyperparameters across the entire dataset. Finally, we present the performance of the BNN and NN whilst continuously reducing the size of the training data to compare their respective performances across a smaller training dataset.

4.1 Hyperparameter Optimisation

Across the BNN and the NN models, we performed hyperparameter optimisation for the learning rate of the optimiser and also the α term in equation (2.11). The following is a summary of the Recall and F_1 Score for both the BNN and the NN using different learning rate and α values:

Table 1: BNN performance for different learning rates and KL divergence weights.

Learning Rate	KL Weight (α)	Recall (%)	F1 Score
0.1	0.1	94.1176	0.9600
0.1	0.01	92.1569	0.9400
0.1	0.001	94.1176	0.9143
0.1	0.0001	92.1569	0.9495
0.01	0.1	100.0000	0.9714
0.01	0.01	96.0784	0.9608
0.01	0.001	100.0000	0.9808
0.01	0.0001	98.0392	0.9615
0.001	0.1	0.0000	0.0000
0.001	0.01	82.3529	0.8936
0.001	0.001	66.6667	0.6869
0.001	0.0001	98.0392	0.8772
0.0001	0.1	98.0392	0.8197
0.0001	0.01	13.7255	0.2373
0.0001	0.001	84.3137	0.7414
0.0001	0.0001	0.0000	0.0000

Table 2: NN performance for different learning rates

Learning Rate	Recall (%)	F_1 Score
0.1	96.0784%	0.9608
0.01	96.0784%	0.9703
0.001	94.1176%	0.8807
0.0001	100.0000%	0.7786

From table 1 we choose the values for the learning rate and α as 0.01 and 0.001, respectively. From table 2 we set the learning rate equal to 0.01. After obtaining the optimal hyperparameters for both the BNN and NN, we once again train and validate the models using these optimal values and evaluate them on the test set.

4.2 Preliminary Results

To split the data into training and testing data, we used *train_test_split* a function from *scikit-learn* which splits the data set into training and testing data. Setting a random seed enables the training and testing to ensure consistent splits across runs and to achieve fairness for both models because the models encounter the same split of training and testing data. The validation set was the last 20% of the full training set and the new training set was the initial 80%. Using the findings in the previous section, we managed to generate two figures showing the training loss and validation loss for each model:

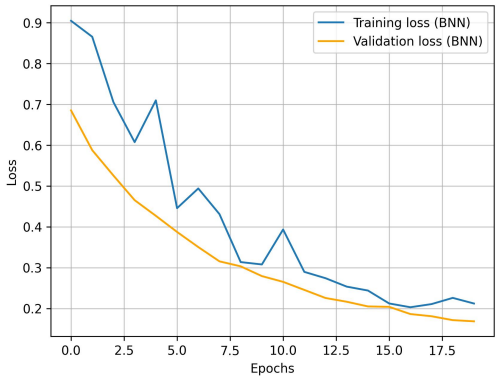

Figure 3: Training and validation loss plot for BNN

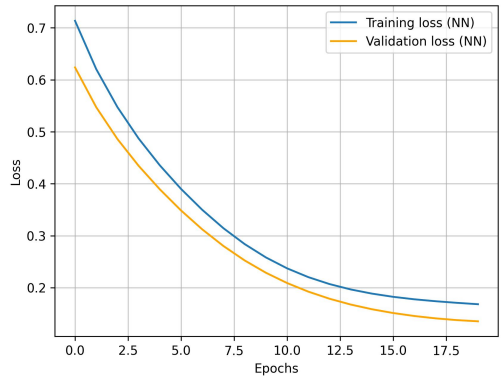

Figure 4: Training and validation loss plot for NN

From figure 3 we see that the training loss has a zig-zag like motion whereas the validation loss is a smoother line. The learning rate and α term in equation (2.11) affects the training loss. These values can further be optimised to smooth the curve however, other values were not considered in the hyperparameter optimisation section. Figure 4 displays both the training loss and validation loss decreasing steadily and smoothly. Both the BNN and NN plateau at epoch 20, and neither show signs of overfitting.

The next figure shows the combined ROC-AUC plot for both the BNN and NN:

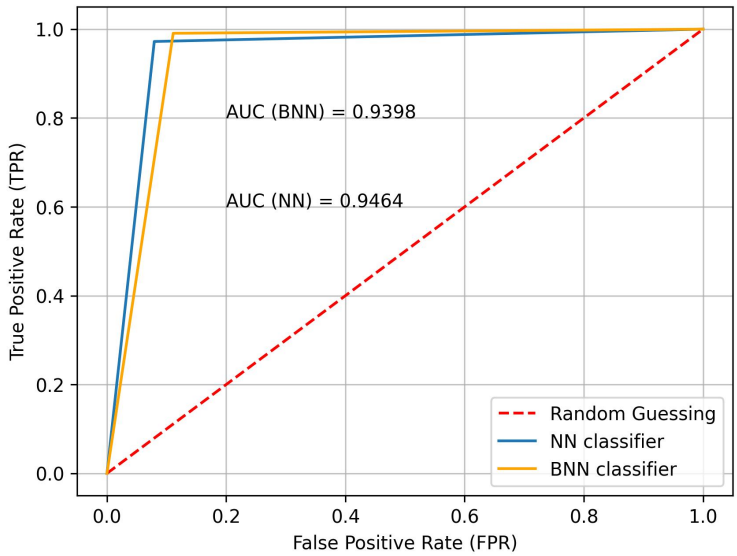

Figure 5: Combined ROC-AUC plot for BNN and NN.

Figure 5 shows the classification ability for both models. Ultimately, the larger the AUC value the better the classification ability. From figure 5, the NN performs slightly better than the BNN however, both the NN and BNN perform significantly better than random guessing. An advantage of a BNN over a NN is that it can quantify the uncertainty in its predictions, as shown in the following table:

Table 3: Mean BNN uncertainty.

Uncertainty in correct predictions	Uncertainty in incorrect predictions
0.208	0.618

The physical interpretation of table 3 is that the BNN is more confident when its predictions are correct and less confident when it is incorrect.

4.3 Final Results

This section compares the performance of the BNN and NN as the training dataset size is continuously reduced. In this setup, the validation set grows in size, whereas the test set size remains fixed at 30% of the original dataset. In effect, the models are provided with fewer training instances before being assessed on new data. The following table compares the Recall, F_1 Score and validation loss of both models using different training set sizes:

Table 4: Performance results for BNN and NN

	BNN						**NN**					
Training size	Recall	F_1 Score	Tra Loss (Epo 19)	Tra Loss (Epo 20)	Val Loss (Epo 19)	Val Loss (Epo 20)	Recall	F_1 Score	Tra Loss (Epo 19)	Tra Loss (Epo 20)	Val Loss (Epo 19)	Val Loss (Epo 20)
0.7	0.9815	0.9636	0.1596	0.1653	0.2082	0.2075	0.9815	0.9636	0.1568	0.1542	0.2236	0.2218
0.6	0.9815	0.9593	0.1768	0.1780	0.1955	0.1887	0.9907	0.9727	0.1434	0.1420	0.1894	0.1896
0.5	0.9815	0.9593	0.1709	0.1921	0.1836	0.1783	0.9627	0.9674	0.1519	0.1493	0.1669	0.1669
0.4	0.9907	0.9596	0.2187	0.1670	0.2153	0.2128	0.9815	0.9770	0.1216	0.1184	0.1978	0.1990
0.3	0.9815	0.9593	0.1588	0.1522	0.1998	0.1952	0.9907	0.9683	0.0986	0.0943	0.1974	0.1998
0.2	0.9907	0.9596	0.1890	0.1671	0.2080	0.2069	0.9907	0.9640	0.1025	0.0976	0.2133	0.2135
0.1	0.9615	0.9636	0.0811	0.0821	0.2206	0.2151	0.9815	0.9464	0.0373	0.0322	0.2211	0.2257
0.05	0.9815	0.9593	0.1179	0.1199	0.2225	0.2169	0.9722	0.9417	0.0437	0.0381	0.2024	0.2039

From table 4 the BNN and NN in terms of their respective Recall and F_1 Scores perform very similarly to all training set sizes. The important thing to note is how the validation loss differs in both models. BNNs are generally less prone to overfitting (unless the implementation is bad) due to the KL divergence term acting as a regularizer in equation (2.11) but are not immune to overfitting. The NN quickly overfits as the training set size becomes smaller, whilst the BNN shows fewer signs of overfitting. We can see signs of overfitting by comparing the training loss (column *Tra Loss* in table 4) and the validation loss (column *Val Loss* in table 4), essentially, if training loss decreases but validation loss increases then, the model is learning the training data too much and is generalising poorly. The *Breast Cancer* dataset is not large and hence, the validation loss and training loss can be quite small regardless if overfitting is taking place or not. If the dataset was larger, the signs of overfitting would be more apparent in the validation and training loss values.

Table 4 shows, if the training set size decreases from 70% to 60% both models perform well and there are no signs of overfitting. However, at 50% training set size, the NN begins to show early signs of overfitting, as the training loss decreases from epoch 19 to epoch 20 while the validation loss stops improving over the same interval. From a training set size of 40%, the NN starts overfitting because there is a noticeable increase in the validation loss but the training loss is decreasing. On the other hand, the BNN for all tested training set sizes shows the training loss decreases whilst the validation loss is also decreasing. The BNN is learning useful patterns and is generalising well to unseen data. Due to the size of the dataset the Recall and F_1 Scores are valued well for all training set sizes however, if the dataset had more instances then perhaps we would see these metrics decreasing.

5 Conclusion

This project has demonstrated the key differences between BNNs and NNs, from their compositions to their optimisation principles and their performance under constrained data environments. The results show that BNNs are particularly advantageous with small datasets, exhibiting greater robustness to overfitting. While NNs can perform well with ample data, their power must be balanced against the dataset's complexity. Given the limited size and less complex nature of the dataset used in this project, the BNNs inherent characteristics provided a distinct advantage against

the NNs. This highlights a fundamental trade-off between predictive performance and uncertainty quantification, where BNNs provide more informative predictions in data-scarce settings.

5.1 Limitations

During this project the *Breast Cancer Wisconsin* dataset was used to train, evaluate and test the models. This dataset appears to be relatively simple, as both models perform well based on their respective Recall and F_1 scores, irrespective of whether overfitting is present or not. This suggests that the classification boundary may not be highly complex. Additionally, as the BNN relies on variational inference to approximate the posterior distribution, the choice of variational family is important and the number of MC samples used during predictions will influence the final predictions.

5.2 Future Work

Future work could explore the use of a more complex dataset to further evaluate the performance difference between BNNs and NNs. Furthermore, the use of different variational families could be explored to see their influence on BNNs and alternative inference methods, such as MCMC, could be explored to improve the approximation of the predictive distribution. Additionally, a comparison between Bayesian Convolutional Neural Networks (BCNNs) and standard Convolutional Neural Networks (CNNs) on image data could be a natural extension of this project.

AI Declaration

I Cornelius Jacobus (CJ) Wessels publicly declare the use of AI for the following tasks:

- AI was used as a debugger in my code and helped with the implementation of some concepts.
- AI was used as a quick way to acquire information about the Python packages *PyTorch* and *torchbnn*.
- AI was used to acquire quick access to sites whose information was used to explain concepts within this project.
- AI was used as a spell checker and helped to reword some sentences to sound more formal.

References

Cleveland Clinic. Tumor (neoplasm): Types, symptoms & treatment, 2024. URL https://my.clevelandclinic.org/health/diseases/21881-tumor. Accessed: 16 March 2026.

David Duvenaud. Bayesian neural networks. *Distill (Draft)*. URL https://www.cs.toronto.edu/~duvenaud/distill_bayes_net/public/.

Chris Hughes. A brief overview of cross entropy loss, 7 2025. URL https://medium.com/@chris.p.hughes10/a-brief-overview-of-cross-entropy-loss-523aa56b75d5.

Gaurav Mittal. An intuitive treatment of negative log-likelihood, cross entropy, kl divergence, and importance sampling, 8 2025. URL https://medium.com/data-science-collective/an-intuitive-treatment-of-negative-log-likelihood-cross-entropy-and-kl-divergence-553f2ea25f07.

Jacopo Talpini. An introduction to bayesian neural network and uncertainty quantification in deep learning. Presentation at Inverted CERN School of Computing, 2023. URL https://indico.cern.ch/event/1208723/contributions/5230073/attachments/2600859/4521507/An%20Introduction%20to%20Bayesian%20Neural%20Network%20and%20Uncertainty%20Quantification%20in%20Deep%20Learning-Jacopo%20Talpini.pdf.